

SPRING BOOT & MICROSERVICE NOTES



SPRING BOOT & MICROSERVICE NOTES

SPRING BOOT & MICROSERVICE

DAY: 7

TOPICS: SPRING BOOT

- 📁 Overview of Spring Boot, Features and Benefits, Getting Started with Spring Boot
- 📁 Setting up a Spring Boot project with Java and starter dependencies.
- 📁 Creating starter project with maven and Spring Boot current version, Understanding Spring boot annotation, `@SpringBootApplication`, Externalizing Configuration using `application.properties` and `application.yml`
- 📁 Introduction to RESTful webservices, Setting up Spring boot project for REST, Creating RESTful Controllers, `@RestController`, `@Service`, `@Repository`, Mapping Requests with `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@DeleteMapping`, `@PutMapping`, Request and Response handling, `@RequestBody`, `@ResponseBody`, `ResponseEntity`, HTTP methods and Status Codes, Best practices for creating RESTful services
- 📁 Build a simple REST API with different HTTP methods and test the endpoints with POSTMAN

Introduction

In traditional Spring applications, developers commonly need to:

- Configure the Spring container.
- Configure dependencies.
- Configure a web server.
- Configure application components.
- Create XML or Java configuration.
- Manage many dependencies manually.

Spring Boot simplifies these tasks and allows developers to create applications quickly with **minimum configuration**.

Overview of Spring Boot

Spring Boot is a framework built on top of the **Spring Framework** that makes it easier to create, configure, and run Spring-based applications.

Spring Framework provides many powerful features, but traditionally developers need to configure several components manually.

Spring Boot reduces this configuration by providing:

- **Auto-configuration**
- **Starter dependencies**
- **Embedded servers**
- **Production-ready features.**

SPRING FRAMEWORK VS SPRING BOOT

Spring Framework	Spring Boot
Requires more configuration	Requires less configuration
External server commonly required for web applications	Embedded server available
Dependencies are managed individually	Starter dependencies simplify dependency management
Manual configuration is common	Auto-configuration is provided
More setup required	Quick application setup
Suitable for highly customized applications	Suitable for rapid development

Features of Spring Boot

I. Auto-Configuration

Spring Boot automatically configures application components based on:

- Dependencies available in the project
- Application configuration
- Spring Boot conventions

For example, if Spring Web dependencies are added, Spring Boot automatically provides many configurations required for a web application.

Benefit

Developers don't have to configure every component manually.

II. Starter Dependencies

Spring Boot provides **Starter dependencies** that group commonly required dependencies together.

STARTER	PURPOSE
SPRING-BOOT-STARTER-WEB	Web and REST applications
SPRING-BOOT-STARTER-DATA-JPA	Spring Data JPA
SPRING-BOOT-STARTER-JDBC	JDBC applications
SPRING-BOOT-STARTER-TEST	Testing
SPRING-BOOT-STARTER-SECURITY	Spring Security

Example

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Instead of adding multiple web-related dependencies individually, the starter provides the commonly required dependencies.

III. Embedded Server

Spring Boot provides embedded servers for web applications.

Common embedded servers include:

- Tomcat
- Jetty
- Undertow

By default, Spring Boot web applications commonly use **Tomcat**.

Traditional Approach

- In Traditional Approach we need to do the following things:
 - Application
 - War Files
 - Add External Server
 - And Finally Run the Application

Spring Boot Approach

- In Spring Boot we need to do the following things:
 - Spring Boot Application
 - Embedded Tomcat Server
 - Run the Application

Note : Therefore, developers generally do not need to install and configure a separate Tomcat server for a typical Spring Boot web application.

IV. Standalone Applications

- Spring Boot applications can be run as standalone Java applications.

```
public static void main(String[] args) {
    SpringApplication.run(Application.class, args);
}
```

- The main() method starts the Spring application.
- This makes application development and deployment simpler.

V. Production-Ready Features

- Spring Boot provides features that help developers monitor and manage applications in production.

One important component is **Spring Boot Actuator**.

Actuator provides endpoints for monitoring information such as:

- Application health

- Application metrics
- Environment information
- Application mappings
- Runtime information

VI. Externalized Configuration

- Spring Boot allows configuration values to be kept outside Java source code. Common configuration files are:

application.properties or application.yml

Example

```
server.port=8081
spring.application.name=StudentApplication
```

This allows configuration to be changed without modifying the Java source code.

VII. Convention Over Configuration

Spring Boot follows the principle:

Use sensible default configurations unless the developer wants to customize them.

For example, developers do not need to configure every Spring component manually. Spring Boot provides reasonable defaults and allows developers to override them when necessary.

Benefits of Spring Boot

1. Faster Development

Spring Boot reduces the amount of configuration required.

2. Less Boilerplate Configuration

Developers can focus more on business logic instead of configuration.

3. Easy Dependency Management

Starter dependencies simplify dependency selection.

4. Embedded Server

Applications can run without manually deploying them to an external server.

5. Easy REST API Development

Spring Boot makes it easy to create RESTful web services using annotations such as:

@RestController
 @GetMapping
 @PostMapping
 @PutMapping
 @DeleteMapping

6. Production Support

Spring Boot Actuator provides monitoring and management capabilities.

7. Easy Integration

Spring Boot integrates well with technologies such as:

- Spring Data JPA
- JDBC
- Spring Security
- Spring MVC
- Spring Cloud
- MySQL
- MongoDB
- Redis

Getting Started with Spring Boot

There are several ways to create a Spring Boot project.

A commonly used approach is **Spring Initializr**.

Step 1: Create a Project

Use Spring Initializr to generate the project structure.

Select:

- Project: Maven
- Language: Java
- Spring Boot version
- Java version
- Required dependencies

For a basic web application, select: Spring Web

Step2: Project Structure

Step3: Main Application Class

```
package com.iits.main;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentApplication {

    public static void main(String[] args) {
        SpringApplication.run(StudentApplication.class, args);
    }
}
```

Understanding @SpringBootApplication

@SpringBootApplication = @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan

1. @SpringBootConfiguration

Indicates that the class provides Spring Boot configuration.

2. @EnableAutoConfiguration

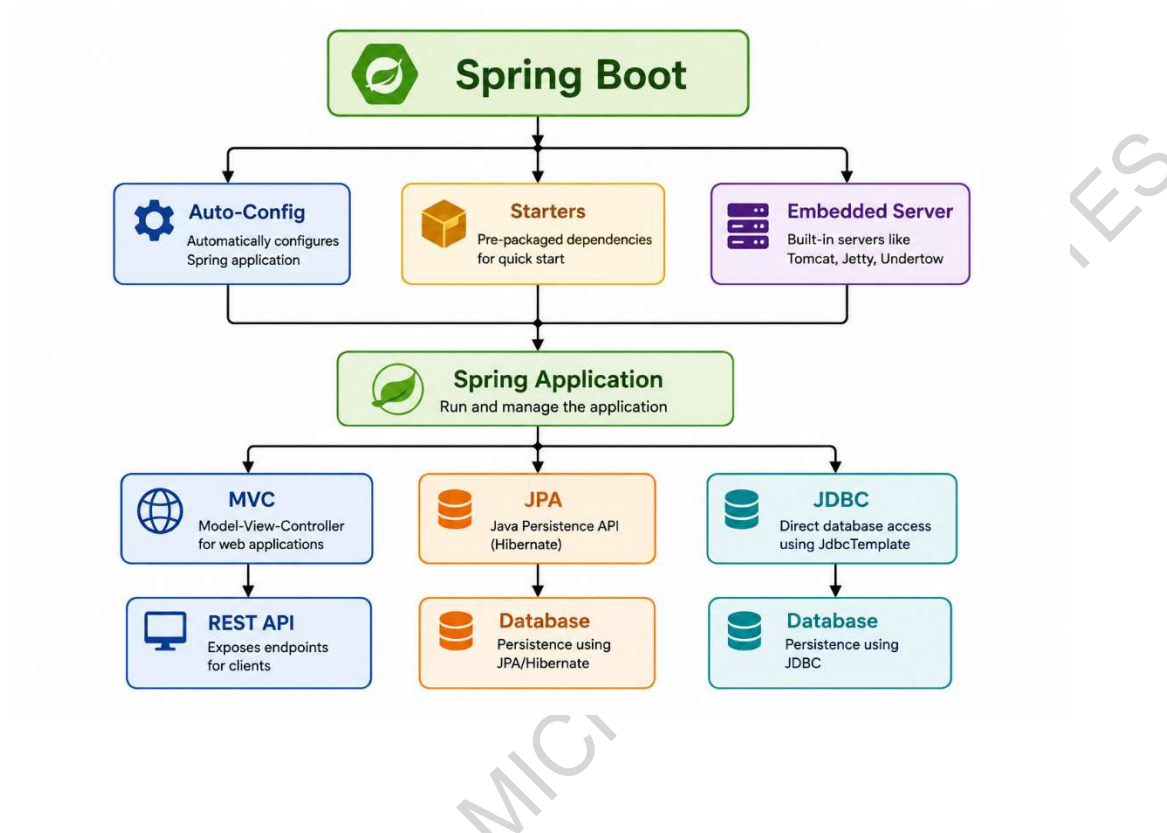
Enables Spring Boot's automatic configuration mechanism.

3. @ComponentScan

Scans the specified package and its subpackages for Spring components such as:

```
@Component
@Service
@Repository
@Controller
@RestController
```

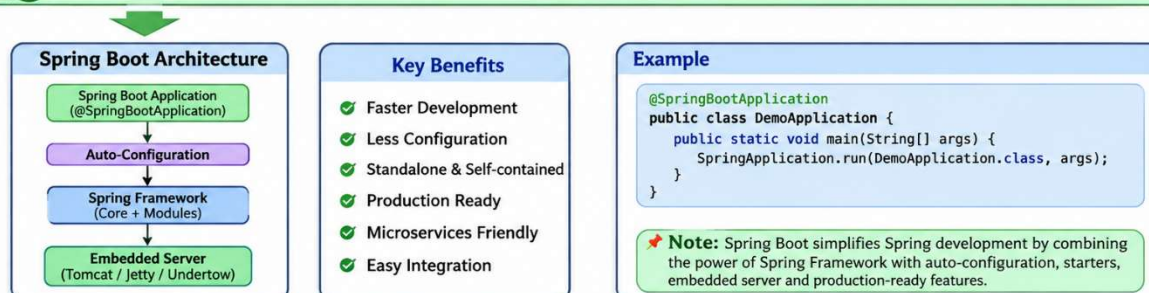

Spring Boot starts the application context and, for a web application, starts the embedded server. A typical console message indicates that the application has started successfully.



Spring Boot = Spring Framework + Auto-Configuration + Starters + Embedded Server + Production



Result: Spring Boot = Rapid Application Development + Convention over Configuration + Production Ready Applications



Part-2

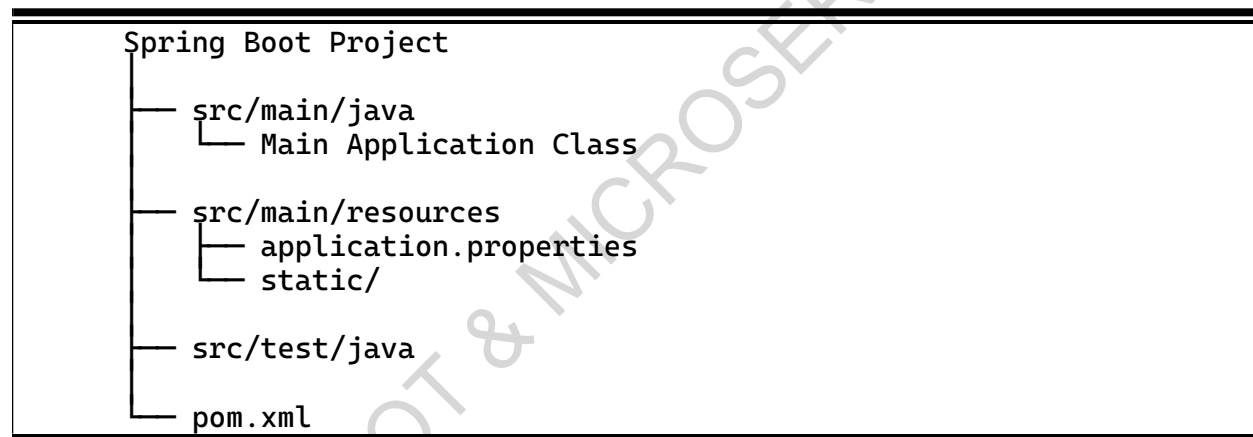
Creating starter project with maven and Spring Boot current version

Understanding Spring boot annotation, @SpringBootApplication, Externalizing Configuration using application.properties and application.yml

What is a Spring Boot Project?

A **Spring Boot project** is a Java application built using Spring Boot that provides automatic configuration, starter dependencies, and an embedded server to simplify Spring application development.

A typical Spring Boot project contains:



Prerequisites

Before creating a Spring Boot project, install:

Requirement	Purpose
JDK	Compile and run Java applications
IDE	IntelliJ IDEA / Eclipse / Spring Tool Suite
Maven	Manage dependencies and build the project
Spring Boot	Develop Spring-based applications

Creating the Project

The easiest approach is **Spring Initializr**.

Open:

[Spring Initializr](#)

What is a Starter Dependency?

A **Spring Boot Starter** is a convenient dependency descriptor that brings together the libraries commonly required for a particular type of application.



Note : Instead of adding many individual dependencies, we can add one starter.

Common Spring Boot Starters

Starter	Purpose
spring-boot-starter-web	Web applications and REST APIs
spring-boot-starter-data-jpa	JPA and database operations
spring-boot-starter-jdbc	JDBC database access
spring-boot-starter-security	Application security
spring-boot-starter-test	Testing
spring-boot-starter-validation	Bean validation
spring-boot-starter-thymeleaf	Server-side HTML applications
spring-boot-starter-actuator	Monitoring and production management

Now let's see how we can do it practically.

What is application.yml?

application.yml is a **configuration file** used in Spring Boot applications.

It is used to define settings such as:

- Server port
- Database configuration
- Application name
- Logging
- Profiles
- JPA/Hibernate settings
- Custom application properties

Spring Boot normally looks for this file inside:

```
src
└─ main
    └─ resources
        └─ application.yml
```

Why do we need application.yml?

Suppose we want our Spring Boot application to run on port 8081.

Instead of writing the port directly in Java code, we put it in the configuration file:

```
server:
  port: 8081
```

Now Spring Boot reads this configuration when the application starts.

application.yml

1. Spring Boot reads configuration
2. Application starts
3. Server runs on port 8081

YAML Meaning

YAML stands for:

YAML Ain't Markup Language

It is a human-readable format commonly used for configuration.

A YAML file generally uses:

- indentation
- key: value
- hierarchical/nested structure

```
server:
  port: 8081
```

application.properties vs application.yml

The same configuration can be written in two ways.

application.properties

application.yml

<pre>server.port=8081 spring.application.name=student-service</pre>	<pre>server: port: 8081 spring: application: name: student-service</pre>
---	---

Note : The YAML version is often easier to read when there are many related properties.

```
name: Imtiaz
age: 25
city: Hyderabad

student:
  name: Imtiaz
  age: 25
  city: Hyderabad
```

Spring Boot Example

```
spring:
  application:
    name: student-service

server:
  port: 8081
```

Database Configuration

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/studentdb
    username: root
    password: root
    driver-class-name: com.mysql.cj.jdbc.Driver
```

```

spring
├── datasource
│   ├── url
│   ├── username
│   ├── password
│   └── driver-class-name

```

Spring Boot uses these properties to configure the DataSource.

JPA Configuration

We can also configure JPA/Hibernate:

```

spring:
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

```

`ddl-auto: update`

Automatically updates database tables based on entities

`show-sql: true`

Displays SQL queries in the console

Example

```

spring:
  application:
    name: student-service

  datasource:
    url: jdbc:mysql://localhost:3306/studentdb
    username: root
    password: root

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true

server:
  port: 8081

```

Lists in YAML

YAML makes lists very readable.

```
students:
  - id: 101
    name: Rahul
    salary: 30000

  - id: 102
    name: Priya
    salary: 35000
```

Important YAML Rule — Indentation

This is **very important**.

```
server:
  port: 8081
```

Incorrect

```
server:
port: 8081
```

Comments

- We can add comments using #.

```
# Server configuration
server:
  port: 8081

# Application configuration
spring:
  application:
    name: student-service
```

Final Example

```
spring:
  application:
    name: student-management

  datasource:
    url: jdbc:mysql://localhost:3306/studentdb
    username: root
    password: root
```

```
jpa:  
  hibernate:  
    ddl-auto: update  
    show-sql: true  
  
server:  
  port: 8081
```

SPRING BOOT & MICROSERVICE NOTES

Part-3

Introduction to RESTful webservice

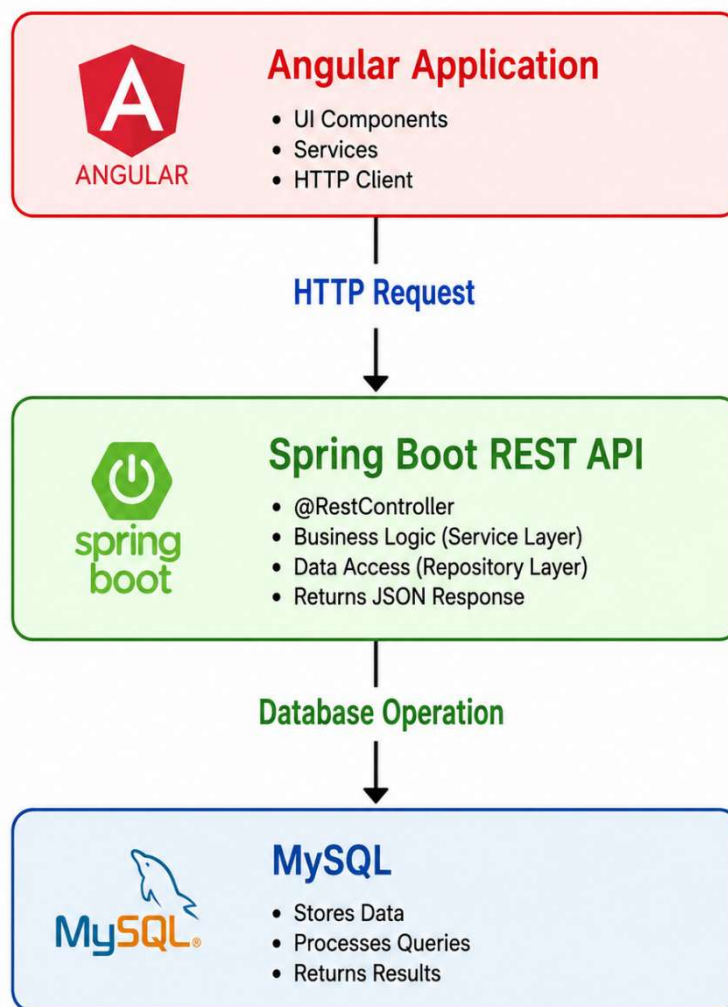
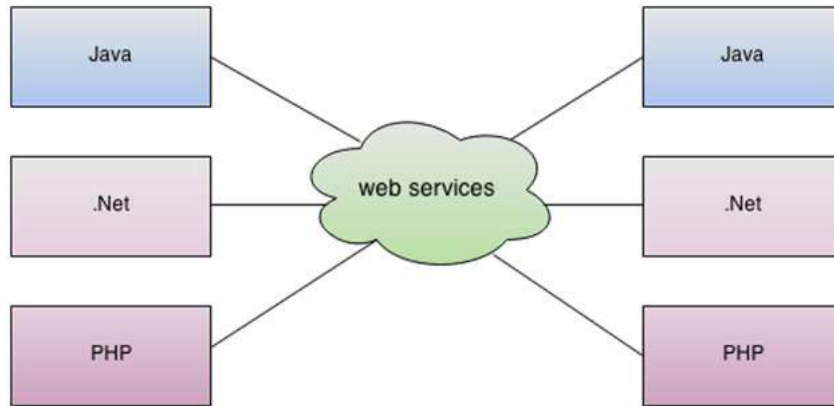
Setting up Spring boot project for REST

Creating RESTful Controllers, @RestController, @Service, @Repository, Mapping Requests with @RequestMapping, @GetMapping, @PostMapping, @DeleteMapping, @PutMapping, Request and Response handling, @RequestBody, @ResponseBody, ResponseEntity, HTTP methods and Status Codes, Best practices for creating RESTful services

SPRING BOOT & MICROSERVICE NOTES

Introduction to RESTful Web Services

A **Web Service** is a software component that allows two different applications to communicate over a network.



REST

- **REST** stands for **Representational State Transfer**.
- REST is an architectural style used to design web services that communicate using HTTP.
- A RESTful service exposes **resources** through URLs.
- For example, consider an Employee application:
-

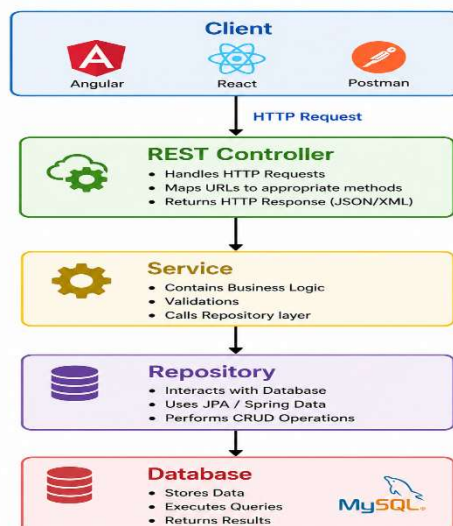
Resource	URL
All employees	/employees
One employee	/employees/101
Create employee	/employees
Update employee	/employees/101
Delete employee	/employees/101

The HTTP method tells the server what operation should be performed.

GET	→ Read
POST	→ Create
PUT	→ Update
DELETE	→ Delete

REST Architecture

A typical REST application looks like:



Responsibilities

Controller

- Receives HTTP requests
- Sends HTTP responses
- Handles request mapping

Service

- Contains business logic
- Performs validations/calculations
- Coordinates application operations

Repository

- Communicates with the database
- Performs CRUD operations

Setting Up a Spring Boot Project for REST

A REST application can be created using **Spring Initializr**.

Typical dependencies:

- 1.Spring Web
- 2.Spring Data JPA
- 3.MySQL Driver

For a simple REST API, **Spring Web** is the important dependency.

```
src/main/java
├── com.example.employee
│   ├── EmployeeApplication.java
│   ├── controller
│   │   └── EmployeeController.java
│   ├── service
│   │   └── EmployeeService.java
│   ├── repository
│   │   └── EmployeeRepository.java
│   └── entity
│       └── Employee.java
```

Creating a REST Controller

In Spring Boot, a REST controller is created using:

```
@RestController
public class EmployeeController {

    @GetMapping("/employees")
    public String getEmployees() {
        return "All Employees";
    }
}
```

@RestController

@RestController is a combination of: @Controller + @ResponseBody

It tells Spring that the class handles REST requests and that method return values should generally be written directly to the HTTP response body.

```
@RestController
public class EmployeeController {

    @GetMapping("/employee")
    public Employee getEmployee() {
        return new Employee(101, "John", 50000);
    }
}
```

Response:

```
{
  "id": 101,
  "name": "John",
  "salary": 50000
}
```

HTTP Methods

HTTP Method	Purpose	Example
GET	Read	/employees
POST	Create	/employees
PUT	Update/replace	/employees/101
PATCH	Partial update	/employees/101
DELETE	Delete	/employees/101

@RequestBody

@RequestBody is used to read data from the HTTP request body and convert it into a Java object.

Client Send the Request

```

http://localhost:8080/app/employees
{
  "id": 101,
  "name": "John",
  "salary": 50000
}

```

EmployeeController.java

```

@PostMapping("/employees")
public Employee createEmployee(
    @RequestBody Employee employee) {

    return employee;
}

```

@ResponseBody

@ResponseBody tells Spring to write the return value directly into the HTTP response body.

```

@Controller
public class EmployeeController {

    @ResponseBody
    @GetMapping("/employee")
    public Employee getEmployee() {
        return new Employee(101, "John", 50000);
    }
}

```

Note : With @RestController, @ResponseBody is effectively applied to controller methods by default.

Request and Response Handling

A request can contain:

-  HTTP Method
-  URL
-  Headers
-  Query Parameters
-  Path Variables
-  Request Body

```
POST /employees
Content-Type: application/json

{
  "name": "John",
  "salary": 50000
}
```

Response

The response can contain:

-  Status Code
-  Headers
-  Response Body

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 101,
  "name": "John",
  "salary": 50000
}
```

ResponseEntity

ResponseEntity provides control over the complete HTTP response.

You can control:

- Response body
- HTTP status
- Headers

```
@GetMapping("/employees/{id}")
public ResponseEntity<Employee> getEmployee(
    @PathVariable int id) {

    Employee employee = service.getEmployee(id);

    return ResponseEntity.ok(employee);
}
```

For a resource that does not exist:

```
return ResponseEntity.notFound().build();
```

For creation:

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(employee);
```

Common methods

```
ResponseEntity.ok()
ResponseEntity.status()
ResponseEntity.created()
ResponseEntity.noContent()
ResponseEntity.notFound()
```

HTTP Status Codes

Status	Meaning
200	OK
201	Created
202	Accepted
204	No Content

4xx — Client Error

Status	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
409	Conflict

5xx — Server Error

Status	Meaning
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable

Complete REST Controller Example

```

@RestController
@RequestMapping("/employees")
public class EmployeeController {

    private final EmployeeService service;

    public EmployeeController(EmployeeService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {
        return ResponseEntity.ok(service.getAllEmployees());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployee(
        @PathVariable int id) {

        Employee employee = service.getEmployee(id);

        if (employee == null) {
            return ResponseEntity.notFound().build();
        }

        return ResponseEntity.ok(employee);
    }

    @PostMapping
    public ResponseEntity<Employee> createEmployee(
        @RequestBody Employee employee) {

        Employee saved = service.createEmployee(employee);

        return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(saved);
    }
}

```

```

@PutMapping("/{id}")
public ResponseEntity<Employee> updateEmployee(
    @PathVariable int id,
    @RequestBody Employee employee) {

    Employee updated =
        service.updateEmployee(id, employee);

    return ResponseEntity.ok(updated);
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteEmployee(
    @PathVariable int id) {

    service.deleteEmployee(id);

    return ResponseEntity.noContent().build();
}
}

```

Endpoints:

```

GET    /employees
GET    /employees/101

POST   /employees

PUT    /employees/101

DELETE /employees/101

```

Rules:

1. Use nouns in URLs
/employees

Avoids

```

/getEmployees
/createEmployee
/deleteEmployee

```

The HTTP method already describes the operation.

2. Use plural resource names

Prefer : /employees

Avoid : /employee

3. Use HTTP methods correctly

```
GET    → Retrieve
POST   → Create
PUT    → Update
DELETE → Delete
```

4. Use meaningful HTTP status codes

```
200 OK
201 Created
204 No Content
400 Bad Request
404 Not Found
500 Internal Server Error
```

5. Use ResponseEntity when response control is required

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .body(employee);
```

6. Separate layers

Do not put everything inside the controller.

7. Use DTOs for API contracts when appropriate

Instead of exposing database entities directly:

```
Entity
|-> DTO
|-> REST Response
```

8. Validate incoming data

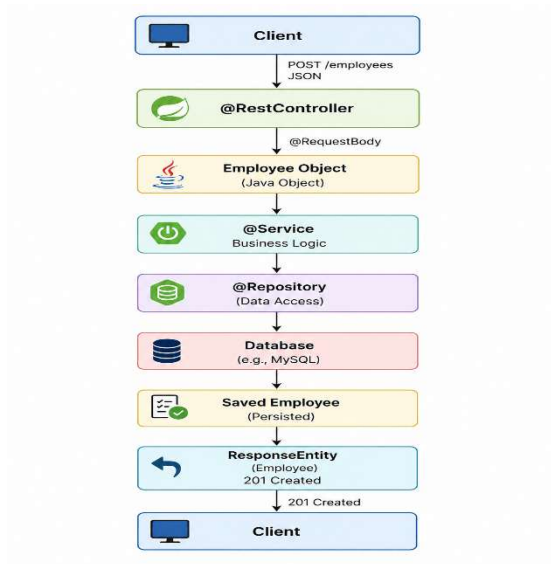
```
@PostMapping
public ResponseEntity<Employee> create(
    @Valid @RequestBody Employee employee) {

    // ...
}
```

9. Handle exceptions centrally

Instead of repeating error handling in every controller, use:

@RestControllerAdvice : For centralized exception handling



All Annotation

Annotation	Purpose
@RestController	Creates REST controller
@RequestMapping	Maps requests
@GetMapping	Handles GET
@PostMapping	Handles POST
@PutMapping	Handles PUT
@DeleteMapping	Handles DELETE
@RequestBody	Converts request body to Java object
@ResponseBody	Writes return value to response body
@PathVariable	Reads value from URL path
@RequestParam	Reads query parameter
@Service	Service/business layer
@Repository	Data access layer
ResponseEntity	Controls HTTP response

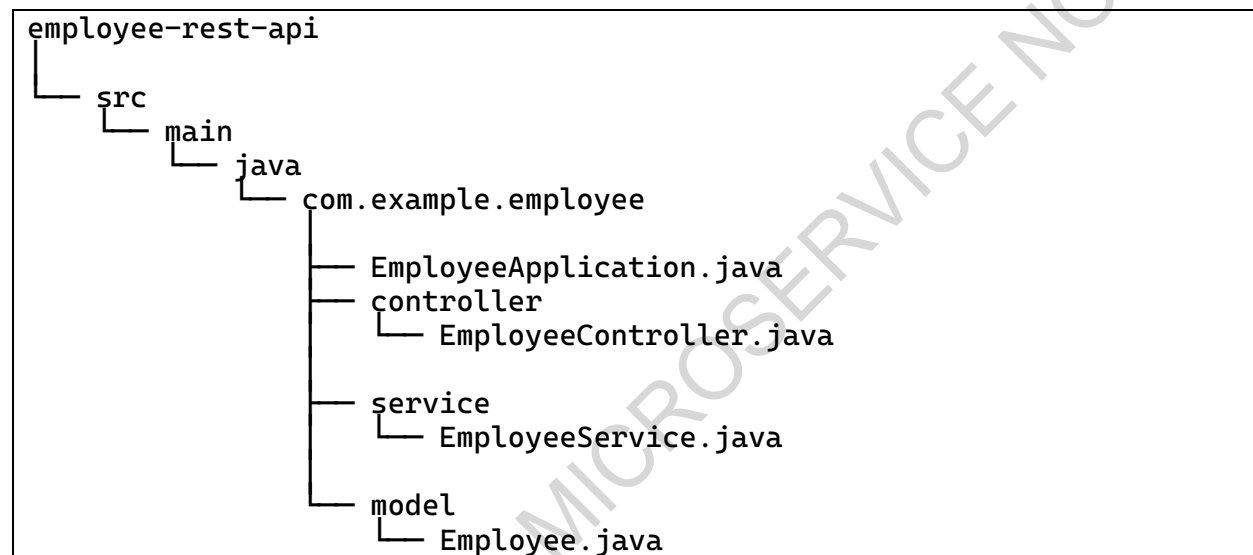
Build a simple REST API with different HTTP methods and test the endpoints with POSTMAN

Step1:

Create Spring Boot Project

Create a Spring Boot project with:

- **Java**
- **Spring Web**
- **Spring Boot DevTools** — optional



Step2: Create Employee Model

```

package com.example.employee.model;

public class Employee {

    private int eid;
    private String ename;
    private double esalary;
    private String dept;

    public Employee() {
    }

    public Employee(int eid, String ename, double esalary, String dept) {
        this.eid = eid;
        this.ename = ename;
        this.esalary = esalary;
        this.dept = dept;
    }
}
  
```

```

    }

    public int getId() {
        return eid;
    }

    public void setId(int eid) {
        this.eid = eid;
    }

    public String getName() {
        return ename;
    }

    public void setName(String ename) {
        this.ename = ename;
    }

    public double getSalary() {
        return esalary;
    }

    public void setSalary(double esalary) {
        this.esalary = esalary;
    }

    public String getDept() {
        return dept;
    }

    public void setDept(String dept) {
        this.dept = dept;
    }
}

```

Step 3: Create Service

```

package com.example.employee.service;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Service;

import com.example.employee.model.Employee;

@Service
public class EmployeeService {

    private List<Employee> employees = new ArrayList<>();
}

```

```

public EmployeeService() {
    employees.add(
        new Employee(101, "Rahul", 45000, "IT")
    );

    employees.add(
        new Employee(102, "Priya", 55000, "HR")
    );

    employees.add(
        new Employee(103, "Amit", 60000, "Finance")
    );
}

// GET all
public List<Employee> getAllEmployees() {
    return employees;
}

// GET by ID
public Employee getEmployeeById(int id) {

    for (Employee employee : employees) {
        if (employee.getId() == id) {
            return employee;
        }
    }

    return null;
}

// POST
public Employee addEmployee(Employee employee) {
    employees.add(employee);
    return employee;
}

// PUT
public Employee updateEmployee(int id, Employee updatedEmployee) {

    for (Employee employee : employees) {

        if (employee.getId() == id) {

            employee.setName(updatedEmployee.getName());
            employee.setSalary(updatedEmployee.getSalary());
            employee.setDept(updatedEmployee.getDept());

            return employee;
        }
    }
}

```

```

    }

    return null;
}

// DELETE
public boolean deleteEmployee(int id) {

    return employees.removeIf(
        employee -> employee.getId() == id
    );
}
}

```

Step4: Create the REST Controller

```

package com.example.employee.controller;

import java.util.List;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.employee.model.Employee;
import com.example.employee.service.EmployeeService;

@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    // GET - All Employees
    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {

        return ResponseEntity.ok(
            employeeService.getAllEmployees()
        );
    }

    // GET - Employee by ID
    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(
        @PathVariable int id) {

```



```

Employee employee =
    employeeService.getEmployeeById(id);

if (employee == null) {
    return ResponseEntity.notFound().build();
}

return ResponseEntity.ok(employee);
}

// POST - Add Employee
@PostMapping
public ResponseEntity<Employee> addEmployee(
    @RequestBody Employee employee) {

    Employee savedEmployee =
        employeeService.addEmployee(employee);

    return ResponseEntity.ok(savedEmployee);
}

// PUT - Update Employee
@PutMapping("/{id}")
public ResponseEntity<Employee> updateEmployee(
    @PathVariable int id,
    @RequestBody Employee employee) {

    Employee updatedEmployee =
        employeeService.updateEmployee(id, employee);

    if (updatedEmployee == null) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(updatedEmployee);
}

// DELETE - Delete Employee
@DeleteMapping("/{id}")
public ResponseEntity<String> deleteEmployee(
    @PathVariable int id) {

    boolean deleted =
        employeeService.deleteEmployee(id);

    if (!deleted) {
        return ResponseEntity
            .notFound()
            .build();
    }
}

```

```

return ResponseEntity.ok(
    "Employee deleted successfully"
);
}
}

```

Stp5: Run the Application

Step6: Test Using Postman

GET All Employees	Method: GET URL: http://localhost:8080/api/employees	[{ "eid": 101, "ename": "Rahul", "esalary": 45000.0, "dept": "IT" }, { "eid": 102, "ename": "Priya", "esalary": 55000.0, "dept": "HR" }, { "eid": 103, "ename": "Amit", "esalary": 60000.0, "dept": "Finance" }]
GET by ID	GET http://localhost:8080/api/employees/101	{ "eid": 101, "ename": "Rahul", "esalary": 45000.0, "dept": "IT" }
POST — Create Employee	http://localhost:8080/api/employees Body → raw → JSON { "eid": 104, "ename": "Sneha", "esalary": 65000, "dept": "IT" }	{ "eid": 104, "ename": "Sneha", "esalary": 65000.0, "dept": "IT" }
PUT — Update Employee	PUT http://localhost:8080/api/employees/104 Body → raw → JSON: { "eid": 104,	{ "eid": 104, "ename": "Sneha Sharma", "esalary": 75000.0, "dept": "Development" }

	<pre>"ename": "Sneha Sharma", "esalary": 75000, "dept": "Development" }</pre>	}
Test DELETE	http://localhost:8080/api/employees/104	200 OK

SPRING BOOT & MICROSERVICE NOTES